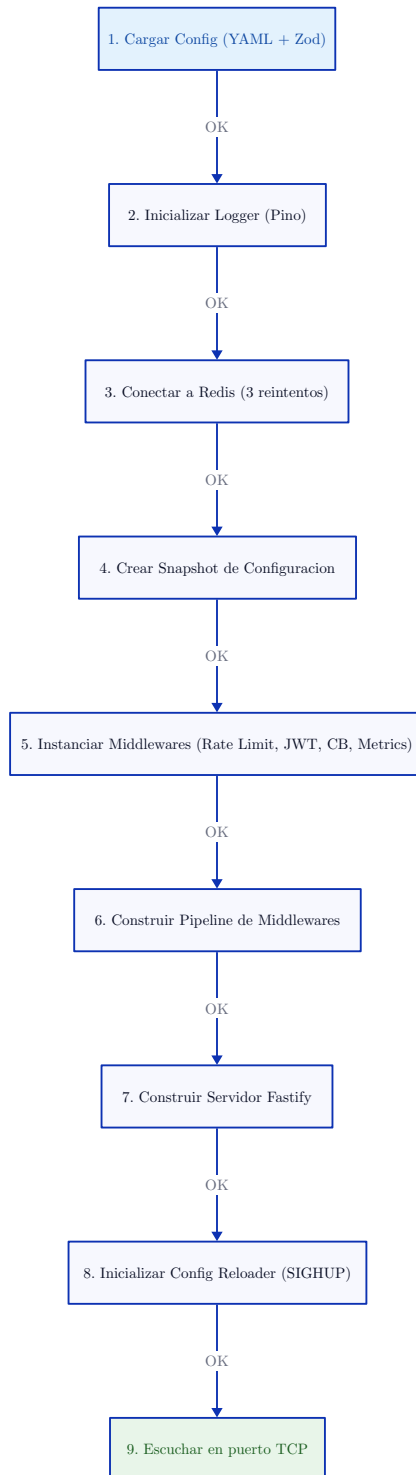


Este documento detalla los archivos de arranque del API Gateway, la secuencia exacta de operaciones que ocurren durante el bootstrap y las configuraciones globales que controlan el comportamiento del sistema.

El Gateway tiene un modelo de ejecución lineal con un único punto de entrada principal. No existe una arquitectura de módulos independientes o workers; todo se orquesta desde un flujo secuencial centralizado.

Entry Point	Ubicacion	Rol Tecnico
index.ts	src/index.ts	Punto de entrada principal. Orquesta el bootstrap completo: carga de configuracion, conexion a Redis, instanciacion de middlewares, construccion del servidor Fastify y registro de signal handlers.
server.ts	src/server.ts	Factoría del servidor. Recibe la configuracion ya validada y construye la instancia Fastify con todas las rutas, hooks, middlewares y el endpoint nativo de metricas.
loader.ts	src/config/loader.ts	Motor de configuracion. Lee el archivo YAML, interpola variables de entorno con sintaxis <code>\\${VAR}</code> , valida contra el esquema Zod y congela el objeto resultante con <code>Object.freeze</code> profundo.
reloader.ts	src/config/reloader.ts	Recarga en caliente. Escucha la señal SIGHUP del sistema operativo y ejecuta un proceso thread-safe de recarga parcial de la configuracion sin reiniciar el proceso.

La funcion `bootstrap()` en `src/index.ts` ejecuta los siguientes pasos en orden estricto. Si cualquier paso falla, el proceso se detiene con `process.exit(1)` para evitar un estado inconsistente.



Detalle de cada paso

1.

Cargar configuracion: `loadConfig()` localiza el archivo YAML (por defecto `./config/gateway.yaml` o la variable de entorno `CONFIG_PATH`), interpola variables de entorno con la sintaxis `\${VARIABLE}`, parsea el YAML con `js-yaml` y valida el resultado contra `GatewayConfigSchema` (Zod). Si falla, lanza una excepcion tipada (`ConfigFileNotFoundError`, `ConfigParseError`, `ConfigValidationError` o `MissingEnvVarError`). El objeto devuelto es inmutable (`Object.freeze` profundo).

1.

Inicializar Logger: Crea una instancia de Pino con el nivel de log de la configuracion (`debug`, `info`, `warn`, `error`). Toda salida es JSON estructurado.

1.

Conectar a Redis: Ejecuta hasta 3 intentos de conexion al servidor Redis configurado. Cada intento tiene un timeout de 2 segundos. Entre intentos espera 1 segundo. Si los 3 fallan, el bootstrap aborta con `process.exit(1)`.

1.

Crear Snapshot: Construye un `ConfigSnapshot` inmutable que contiene la configuracion validada, una instancia de `RouteRegistry` y un timestamp. Este snapshot se almacena en una referencia mutable (`snapshotRef.current`) que permite el hot-reload posterior.

1.

Instanciar Middlewares: Crea las instancias de los 4 plugins del pipeline en este orden:

- `RateLimitPlugin` (respaldado por Redis).
- `JwtAuthPlugin` (verificacion de tokens con `jose`).
- `CircuitBreakerPlugin` (maquina de estados por ruta).
- `MetricsPlugin` (contadores y histogramas Prometheus, condicional a `metrics.enabled`).

1.

Construir Pipeline: `MiddlewarePipeline` recibe el array de plugins y recolecta sus lifecycle hooks para integrarlos con el `ProxyEngine`.

1.

Construir Servidor Fastify: `buildServer()` registra el manejador global de errores, define la propiedad reactiva `routeRegistry`, engancha los hooks de metricas, registra el endpoint `\metrics` y crea las rutas proxy con `preHandler` (pipeline) y `handler` (proxy engine).

1.

Inicializar Config Reloader: Crea una instancia de `ConfigReloader` que reacciona a la senal `SIGHUP` del sistema operativo para recargar la configuracion en caliente sin detener el proceso.

1.

Escuchar en puerto TCP: `server.listen({ port, host })` abre el socket TCP y comienza a aceptar conexiones entrantes.

El Gateway registra handlers para tres senales del sistema operativo:

Senal	Comportamiento	
<code>SIGTERM</code>	Ejecuta un graceful shutdown: cierra el servidor HTTP, drena los connection pools de Undici, cierra la conexion a Redis y sale con codigo 0.	<code>SIGINT</code>
Identico a SIGTERM. Permite apagar limpiamente con Ctrl+C durante el desarrollo.	<code>SIGHUP</code>	Recarga la configuracion en caliente. Ejecuta <code>ConfigReloader.reload()</code> que carga, valida e intercambia atomicamente el snapshot de configuracion activo. Los cambios de <code>logging.level</code> y <code>rateLimit</code> se aplican de inmediato; los de <code>server.port</code> , <code>server.host</code> y <code>redis.url</code> son ignorados (requieren reinicio).

El proceso de apagado seguro (`gracefulShutdown()`) sigue un orden estricto para evitar perdida de datos o conexiones huérfanas:

1. Cerrar el servidor Fastify (dejar de aceptar nuevas peticiones HTTP).
2. Cerrar todos los connection pools de Undici (drenar sockets persistentes hacia backends).
3. Cerrar la conexion a Redis (liberar el socket del store de Rate Limiting).
4. Salir del proceso con `process.exit(0)` .

Si ocurre un error durante el shutdown, el proceso sale con `process.exit(1)` .